

Tài liệu Module AttentionMonitoringSystem

Lê Hoàng An

May 31, 2025

1 Giới thiệu

Tài liệu này mô tả các luồng xử lý của class `AttentionMonitoringSystem`, một lớp tích hợp các thành phần (`FaceDetector`, `FaceFeatureExtractor`, `AttentionClassifier`) để theo dõi trạng thái chú ý của khuôn mặt (Chú ý, Mất tập trung, Buồn ngủ, Vắng mặt, Không chắc chắn, Lỗi xử lý). Class xử lý frame ảnh, ghi nhận lịch sử trạng thái, trực quan hóa kết quả, phân tích hiệu suất trên dataset, và tạo báo cáo chi tiết. Tài liệu bao gồm các luồng chính, vòng đời (lifecycle), luồng xác thực đầu vào, và các luồng chi tiết liên quan đến quá trình theo dõi.

2 Flow Chính của Module

Module `AttentionMonitoringSystem` hoạt động theo các bước chính sau:

1. **Khởi tạo:** Khởi tạo các thành phần `FaceDetector`, `FaceFeatureExtractor`, và `AttentionClassifier` với các tham số cấu hình.
2. **Xử lý frame:** Phát hiện khuôn mặt, trích xuất đặc trưng, phân loại trạng thái, và vẽ kết quả lên frame.
3. **Ghi nhận trạng thái:** Theo dõi lịch sử trạng thái và các chuyển đổi trạng thái.
4. **Phân tích dataset:** Đánh giá hiệu suất hệ thống trên tập dữ liệu với các chỉ số như tỷ lệ phát hiện khuôn mặt, thời gian xử lý, và phân phối trạng thái.
5. **Tạo báo cáo:** Tạo báo cáo chi tiết về lịch sử trạng thái, thời lượng, và điểm tập trung.

6. **Lưu kết quả:** Lưu lịch sử và báo cáo dưới dạng CSV và JSON.

3 Lifecycle của Module

Vòng đời của một đối tượng AttentionMonitoringSystem bao gồm các giai đoạn sau:

1. **Khởi tạo (__init__):**

- Nhận các tham số: face_detector_method, feature_extractor_method, classifier_model_path, và device.
- Khởi tạo FaceDetector với các tham số phát hiện khuôn mặt.
- Khởi tạo FaceFeatureExtractor để trích xuất đặc trưng.
- Khởi tạo AttentionClassifier với mô hình đã lưu hoặc mô hình mặc định.
- Thiết lập các biến theo dõi: trạng thái hiện tại, lịch sử, số lần chuyển đổi, và ngưỡng confidence.

2. **Xử lý frame (process_frame):**

- Phát hiện khuôn mặt, trích xuất đặc trưng, và phân loại trạng thái.
- Ghi nhận trạng thái vào lịch sử.
- Vẽ kết quả lên frame và trả về thông tin chi tiết.

3. **Phân tích dataset (analyze_dataset):**

- Xử lý tập hợp ảnh từ dataset_loader.
- Tính toán các chỉ số hiệu suất và trực quan hóa kết quả.

4. **Tạo báo cáo (generate_report):**

- Tính toán thống kê về trạng thái, thời lượng, và điểm tập trung.
- Lưu lịch sử trạng thái hiện tại trước khi tạo báo cáo.

5. **Lưu kết quả (save_results):**

- Lưu lịch sử trạng thái vào file CSV và báo cáo vào file JSON.

6. **Kết thúc:** Không có phương thức hủy rõ ràng, đối tượng được giải phóng tự động bởi Python garbage collector khi không còn được tham chiếu.

4 Input Validation Flow

Luồng xác thực đầu vào được thực hiện như sau:

1. Khởi tạo (`__init__`):

- Không có kiểm tra rõ ràng cho `face_detector_method`, `feature_extractor_method` hoặc `device`. Các giá trị không hợp lệ có thể gây lỗi trong `FaceDetector` hoặc `FaceFeatureExtractor`.
- Kiểm tra `classifier_model_path` gián tiếp thông qua `AttentionClassifier`, nhưng không kiểm tra tính hợp lệ của file trước khi truyền.

2. Xử lý frame (`process_frame`):

- Kiểm tra frame: Giả định frame là mảng NumPy hợp lệ (được xử lý bởi `FaceDetector`).
- Kiểm tra số lượng khuôn mặt: Nếu không có khuôn mặt, trả về trạng thái Vắng mặt với `confidence 1.0`.
- Kiểm tra đặc trưng: Nếu trích xuất đặc trưng thất bại, trả về trạng thái Lỗi trích xuất.
- Kiểm tra `confidence`: Nếu xác suất dự đoán dưới `confidence_threshold`, trả về trạng thái Không chắc chắn.
- Xử lý lỗi: Bắt ngoại lệ và trả về trạng thái Lỗi xử lý.

3. Phân tích dataset (`analyze_dataset`):

- Kiểm tra `dataset_loader`: Đảm bảo `image_paths` được tải trước khi xử lý.
- Kiểm tra frame: Nếu `cv2.imread` trả về `None`, bỏ qua ảnh.
- Xử lý lỗi: Bắt ngoại lệ trong quá trình xử lý ảnh và ghi log lỗi.

4. Lưu kết quả (`save_results`):

- Kiểm tra `history`: Nếu rỗng, in thông báo và thoát.
- Kiểm tra `output_dir`: Tạo thư mục nếu chưa tồn tại bằng `os.makedirs`.

- Xử lý kiểu dữ liệu: Chuyển đổi các kiểu NumPy thành kiểu Python gốc để đảm bảo tương thích với JSON.

5 Các Flow Chi Tiết

Dưới đây là các luồng chi tiết liên quan đến các phương thức chính:

5.1 Xử lý frame (`process_frame`)

- **Đầu vào:** Frame ảnh (NumPy array).
- **Xử lý:**
 - Ghi nhận thời gian bắt đầu.
 - Gọi `FaceDetector.detect_faces` để phát hiện khuôn mặt.
 - Nếu không có khuôn mặt, gán trạng thái Vắng mặt.
 - Nếu có khuôn mặt:
 - * Gọi `FaceFeatureExtractor.extract_features` để trích xuất đặc trưng.
 - * Nếu trích xuất thất bại, gán trạng thái Lỗi trích xuất.
 - * Gọi `AttentionClassifier.predict` để phân loại trạng thái.
 - * Nếu xác suất dưới `confidence_threshold`, gán trạng thái Không chắc chắn.
 - Ghi nhận trạng thái bằng `record_state`.
 - Vẽ kết quả lên frame bằng `draw_results`.
 - Tính thời gian xử lý.
- **Đầu ra:** Dictionary chứa frame đã xử lý, số khuôn mặt, trạng thái, confidence, và thời gian xử lý.

5.2 Ghi nhận trạng thái (`record_state`)

- **Đầu vào:** Thông tin trạng thái (`state_info`).
- **Xử lý:**

- Nếu `start_time` là `None`, gán thời gian hiện tại.
- Kiểm tra thay đổi trạng thái:
 - * Nếu trạng thái thay đổi, lưu trạng thái trước vào `history` với thời lượng và `confidence`.
 - * Cập nhật trạng thái hiện tại, thời gian bắt đầu trạng thái, và tăng số lần chuyển đổi.
- Lưu `confidence` hiện tại.
- **Đầu ra:** Không có (cập nhật thuộc tính nội bộ).

5.3 Vẽ kết quả (`draw_results`)

- **Đầu vào:** `Frame`, hộp giới hạn, xác suất, thông tin trạng thái.
- **Xử lý:**
 - Tạo bản sao `frame` để tránh sửa đổi bản gốc.
 - Vẽ hộp giới hạn với màu xanh lá (`confidence > 0.7`) hoặc cam.
 - Vẽ `confidence` của hộp giới hạn.
 - Vẽ trạng thái, `confidence`, và số lượng khuôn mặt với màu sắc theo trạng thái.
- **Đầu ra:** `Frame` đã được vẽ kết quả.

5.4 Lấy màu trạng thái (`_get_state_color`)

- **Đầu vào:** Trạng thái.
- **Xử lý:**
 - Trả về màu tương ứng: xanh lá (Chú ý), cam (Mất tập trung), đỏ (Buồn ngủ), xám (Văng mặt), vàng (Không chắc chắn), tím (Lỗi xử lý), trắng (mặc định).
- **Đầu ra:** Tuple màu RGB.

5.5 Phân tích dataset (`analyze_dataset`)

- **Đầu vào:** `dataset_loader`, `sample_size`.

- **Xử lý:**
 - Tải danh sách ảnh từ `dataset_loader`.
 - Lặp qua từng ảnh:
 - * Đọc ảnh bằng `cv2.imread`.
 - * Gọi `process_frame` để xử lý.
 - * Cập nhật thống kê: số ảnh, số ảnh có khuôn mặt, tổng số khuôn mặt, phân phối trạng thái, confidence, và thời gian xử lý.
 - Tính tỷ lệ phát hiện khuôn mặt và confidence trung bình.
 - In báo cáo bằng `_print_analysis_report`.
 - Vẽ biểu đồ bằng `_plot_analysis_results`.
- **Đầu ra:** Dictionary chứa các chỉ số hiệu suất.

5.6 In báo cáo phân tích (`_print_analysis_report`)

- **Đầu vào:** Kết quả phân tích.
- **Xử lý:**
 - In số ảnh xử lý, số ảnh có khuôn mặt, tỷ lệ phát hiện, tổng số khuôn mặt, thời gian xử lý trung bình, confidence trung bình, và phân phối trạng thái.
- **Đầu ra:** Không có (in ra console).

5.7 Vẽ kết quả phân tích (`_plot_analysis_results`)

- **Đầu vào:** Kết quả phân tích.
- **Xử lý:**
 - Vẽ 4 biểu đồ:
 - * Biểu đồ tròn: Phân phối trạng thái.
 - * Biểu đồ histogram: Phân phối thời gian xử lý với đường trung bình.

- * Biểu đồ thanh: Số ảnh xử lý, ảnh có khuôn mặt, và tổng khuôn mặt.
- * Biểu đồ thanh: Tỷ lệ phát hiện, confidence trung bình, và FPS (chuẩn hóa).
- **Đầu ra:** Biểu đồ hiển thị trên màn hình.

5.8 Tạo báo cáo (**generate_report**)

- **Đầu vào:** Không có (sử dụng thuộc tính nội bộ).
- **Xử lý:**
 - Lưu trạng thái hiện tại vào history.
 - Tính tổng thời lượng, thời lượng từng trạng thái, và phần trăm trạng thái.
 - Tính các chỉ số bổ sung: confidence trung bình, tỷ lệ chú ý, số lần mất tập trung, và điểm tập trung.
- **Đầu ra:** Dictionary chứa thông tin báo cáo.

5.9 Tính điểm tập trung (**_calculate_focus_score**)

- **Đầu vào:** Phần trăm trạng thái, confidence trung bình, số lần chuyển đổi.
- **Xử lý:**
 - Tính điểm dựa trên: tỷ lệ Chú ý, phạt cho Mất tập trung và Buồn ngủ, phạt cho số lần chuyển đổi, và thưởng cho confidence cao.
 - Giới hạn điểm trong [0, 100].
- **Đầu ra:** Điểm tập trung (0-100).

5.10 Lưu kết quả (**save_results**)

- **Đầu vào:** Thư mục đầu ra output_dir.
- **Xử lý:**
 - Nếu history rỗng, in thông báo và thoát.

- Tạo thư mục `output_dir` nếu chưa tồn tại.
- Lưu `history` vào file CSV.
- Gọi `generate_report` và lưu báo cáo vào file JSON, chuyển đổi kiểu NumPy thành kiểu Python gốc.
- **Đầu ra:** Không có (lưu file và in thông báo).

6 Kết luận

`ClassAttentionMonitoringSystem` tích hợp các thành phần để tạo thành một hệ thống theo dõi trạng thái chú ý hoàn chỉnh. Các luồng xử lý được thiết kế để đảm bảo tính mạnh mẽ, bao gồm xử lý lỗi, xác thực đầu vào, trực quan hóa kết quả, và tạo báo cáo chi tiết. Để sử dụng hiệu quả, cần đảm bảo các thư viện (`opencv-python`, `facenet-pytorch`, `mediapipe`, `scikit-learn`, `matplotlib`, `seaborn`, `pandas`, `numpy`) được cài đặt và dữ liệu đầu vào (`frame`, `dataset`) được chuẩn bị đúng cách.